AI Engineer Roadmap

1️⃣ What does an AI Engineer actually do?

An **AI Engineer** is not just a “model trainer”.

They mainly:

- Use **existing AI models** (OpenAI, LLMs, ML models)
- Build **real features** using AI
- Integrate AI into **apps, APIs, products**
- Optimize, deploy, and maintain AI systems

Think of it like this:

AI Researcher → creates models
AI Engineer → uses models to build products ✅

You are **already very close** to this role.

2️⃣ Skills you already have (Huge Advantage 💪)

You already know:

Your Skill	How it Helps in AI Engineering
JavaScript / PHP	AI APIs, prompt logic, workflows
Laravel APIs	AI microservices
React / Next.js	AI-powered UI (chat, summaries, dashboards)
REST APIs	Model integration
DB & SQL	Vector DB + AI storage
Healthcare domain	AI use cases (coding, summaries, prediction)

👉 **You don’t need hardcore math or deep ML initially**

3️⃣ AI Engineer Learning Roadmap (Practical Path)

🔹 Phase 1: AI Foundations (2–3 weeks)

You only need **conceptual clarity**, not PhD math.

Learn:

- What is AI vs ML vs DL
- What are **LLMs**
- What is **prompt engineering**
- Tokens, embeddings, context window

✅ Focus: *Understanding, not memorizing*

🔹 Phase 2: Python for AI (Lightweight) 🐍

You don’t need advanced Python.

Learn only:

- Python basics
- Virtual environments
- `requests`, `json`
- Basic data handling

💡 You’ll mostly use Python for:

- Model calls
- AI scripts
- Pipelines

🔹 Phase 3: Core AI Engineer Skills ⭐ (MOST IMPORTANT)

This is where you become an **AI Engineer**.

🔹 1. LLM APIs

Learn how to use:

- OpenAI / Azure OpenAI
- Prompt templates
- System vs user prompts
- Function calling

Example:

Summarize patient notes
Generate medical codes
Explain diagnosis in simple terms

🔹 2. Prompt Engineering (Critical Skill)

You must learn:

- Zero-shot / Few-shot prompts
- Role-based prompts
- Chain-of-thought (controlled)
- Output formatting (JSON)

👉 This is **more valuable than ML math**

🔹 3. RAG (Retrieval Augmented Generation)

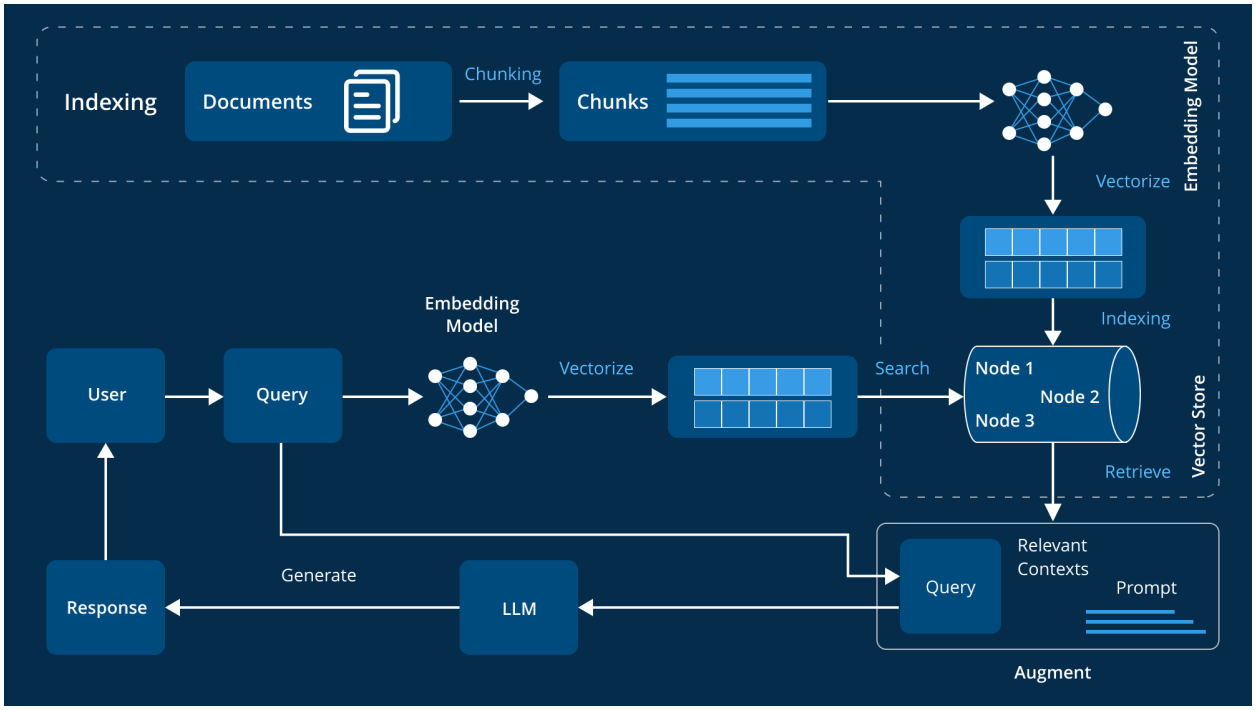
Very important for real apps.

Learn:

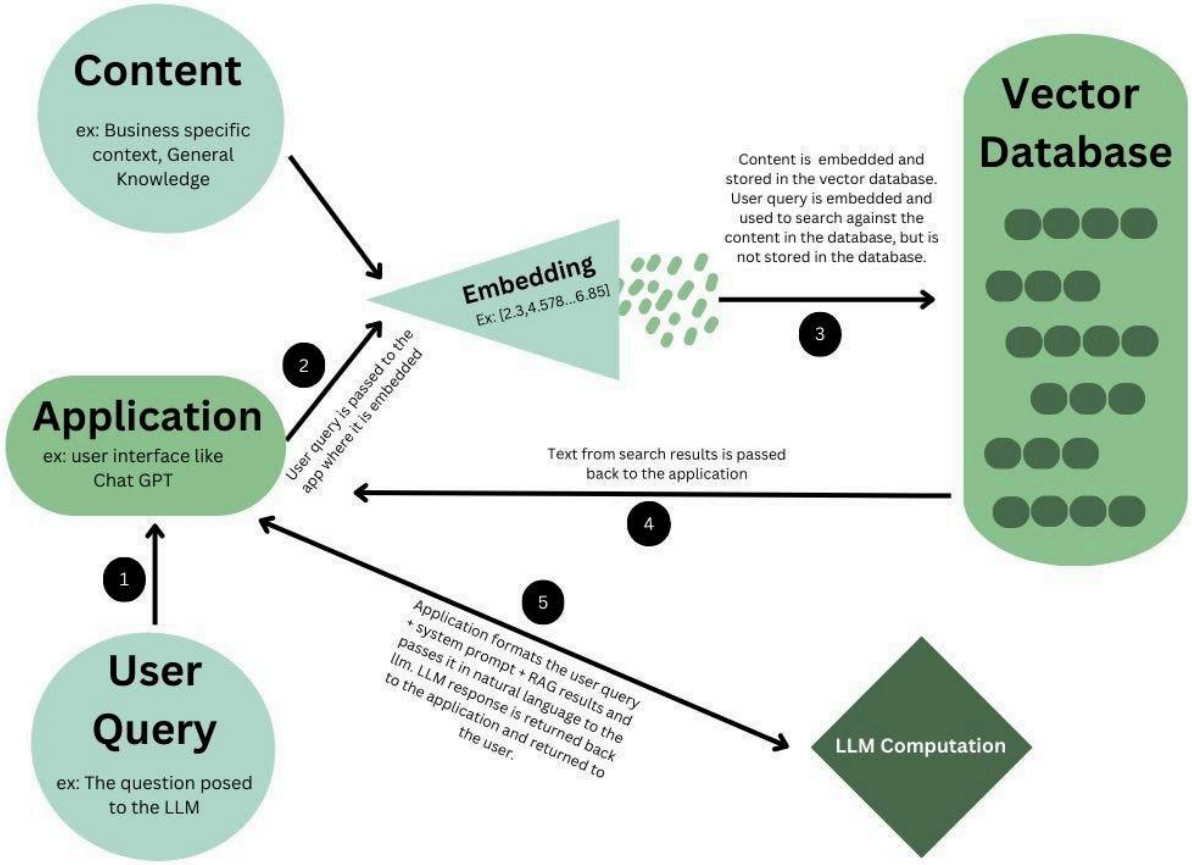
- Embeddings
- Vector databases
- Document search
- Context injection

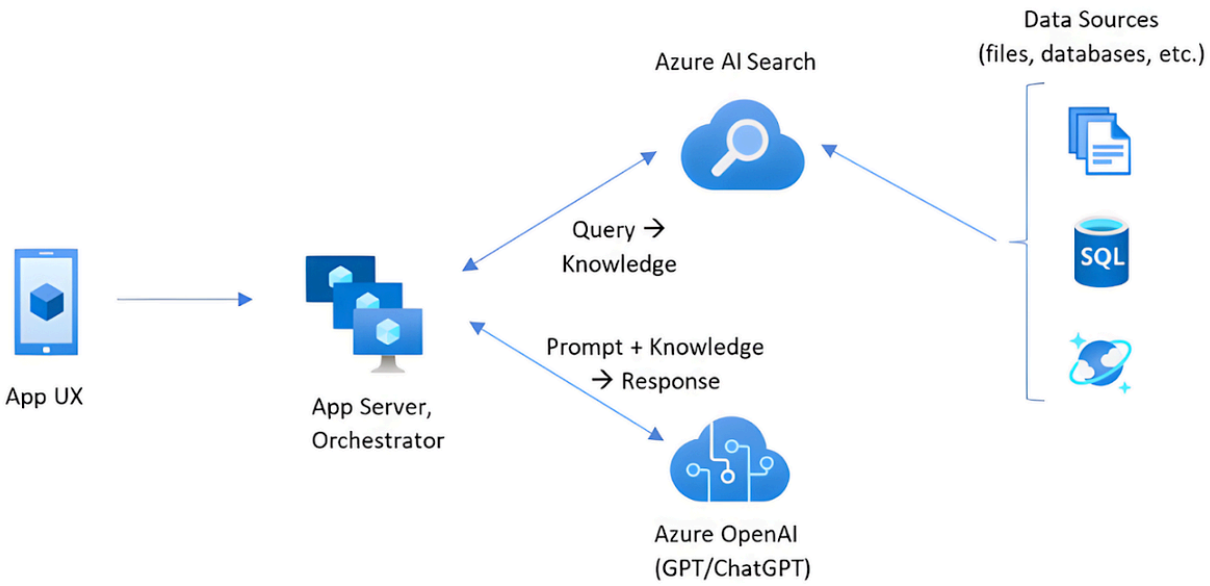
Use cases:

- PDF analysis
- EHR documents
- Internal knowledge base
- Policies & SOPs



created by Sandi Besen





🔹 4. AI + Backend Integration

This is where **you shine** 🔥

Build:

- AI APIs in Laravel / Node
- Rate limiting
- Caching AI responses
- Async jobs
- Logging & monitoring

🔹 Phase 4: Real Projects (Non-Negotiable)

You MUST build projects.

Suggested projects (perfect for your profile):

1️⃣ AI Medical Assistant

- Upload patient notes
- AI summarizes
- Suggest ICD/CPT codes

2️⃣ **AI Document Analyzer**

- Upload PDF
- Ask questions
- Get structured answers

3️⃣ **AI Chatbot for Internal Tools**

- Company policies
- Technical docs
- SOPs

4️⃣ **AI Report Generator**

- Convert raw data → executive summary

These projects = **resume gold**

4️⃣ How to Add AI Engineer Skill (Very Important)

🔹 Resume – Before ❌

Backend Developer | Laravel | React

🔹 Resume – After ✅

AI Engineer | Full-Stack Developer
• Built AI-powered APIs using LLMs
• Implemented RAG for document intelligence
• Integrated AI into healthcare workflows

🔹 Skills Section (Updated)

AI Engineering:
- Prompt Engineering
- LLM Integration (OpenAI/Azure)
- RAG Architecture
- Vector Databases
- AI API Design

Tech:

- Laravel, PHP, JavaScript
- React, Next.js
- REST APIs, SQL

🔹 LinkedIn Headline (Example)

Senior Software Engineer | AI Engineer | Building AI-powered Healthcare Solutions

---

5️⃣ Timeline (Realistic & Safe)

Month	Outcome
Month 1	AI basics + Python + LLM APIs
Month 2	RAG + Prompt Engineering
Month 3	2–3 AI projects
Month 4	Resume + LinkedIn + Job-ready

6️⃣ Truth You Should Know 🧠

- ❌ You don’t need to become a Data Scientist
- ❌ You don’t need advanced ML math
- ✅ You need **product-focused AI skills**
- ✅ AI Engineer = **Software Engineer + AI**

You are **perfectly positioned** for this role.

1️⃣ Daily 60-Day AI Engineer Learning Plan

⏱️ **Daily time**: 1.5–2 hours
🎯 **Goal after 60 days**:

> "I can design, build, and deploy AI-powered applications"

🟢 DAYS 1–10: AI Foundations + LLM Basics

Day 1

- What is AI vs ML vs DL
- What is an LLM
- How ChatGPT works (high level)

Day 2

- Tokens, context window
- Temperature, top-p
- Why prompts matter

Day 3

- Prompt structure:
  - System
  - User
  - Assistant

Day 4

- Zero-shot prompting
- Few-shot prompting

Day 5

- Role-based prompts
- Output control (JSON, tables)

Day 6

- Prompt failures & hallucinations
- Guardrails basics

Day 7

- Use ChatGPT like an engineer
- Prompt versioning

Day 8

- LLM use cases in healthcare & SaaS
- Automation thinking

Day 9

- AI ethics & data privacy (important for healthcare)

Day 10

- Mini task:
  👉 Write prompts for:
  - Summary
  - Code explanation
  - Medical note simplification

🟢 DAYS 11–20: Python + LLM APIs (Lightweight)

You are **not becoming a Python dev**, only AI-friendly Python.

Day 11

- Python basics (variables, loops, functions)

Day 12

- Virtual environments
- pip
- requirements.txt

Day 13

- JSON handling
- HTTP requests

Day 14

- Calling an LLM API (basic)

Day 15

- Prompt templates in code

Day 16

- Error handling for AI calls

Day 17

- Response parsing
- JSON output validation

Day 18

- Token limits & cost control

Day 19

- Retry logic
- Timeout handling

Day 20

- Mini project:
  👉 Python script → input text → AI summary

🟢 DAYS 21–35: Core AI Engineer Skills (MOST IMPORTANT)

🔹 Prompt Engineering (Days 21–25)

Day 21

- Prompt chaining

Day 22

- Context injection

Day 23

- Structured outputs (JSON schema)

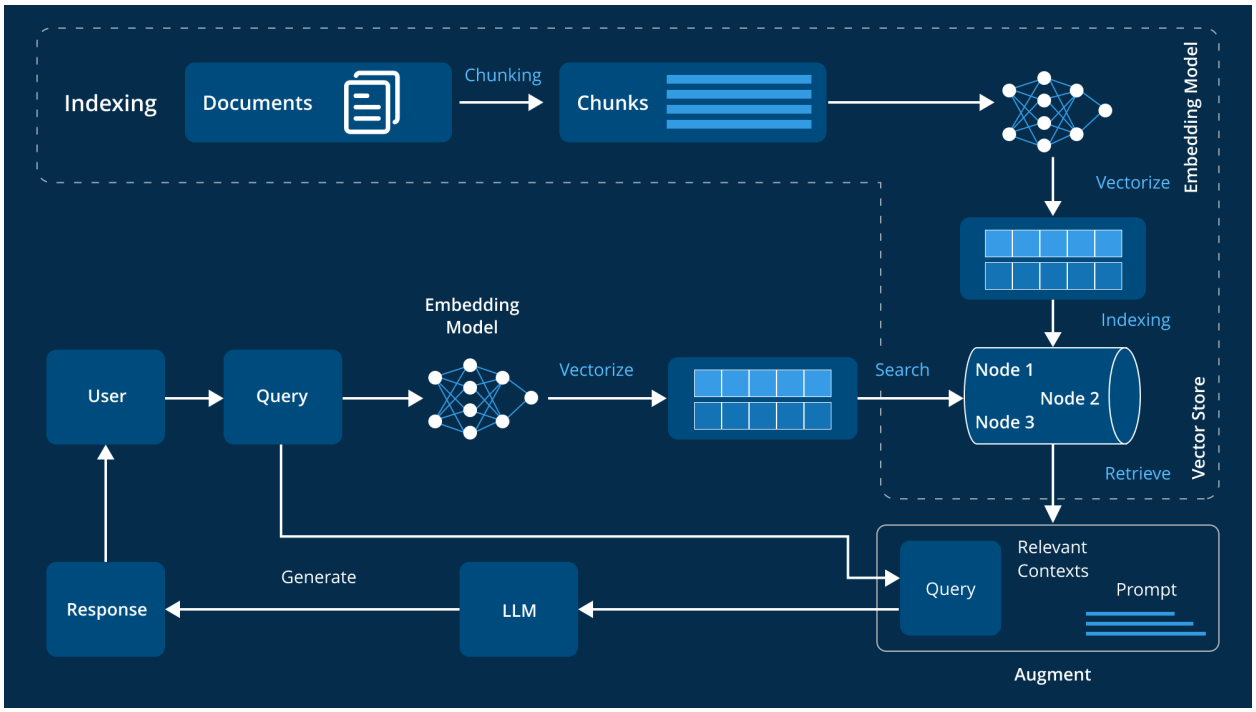
Day 24

- Defensive prompting

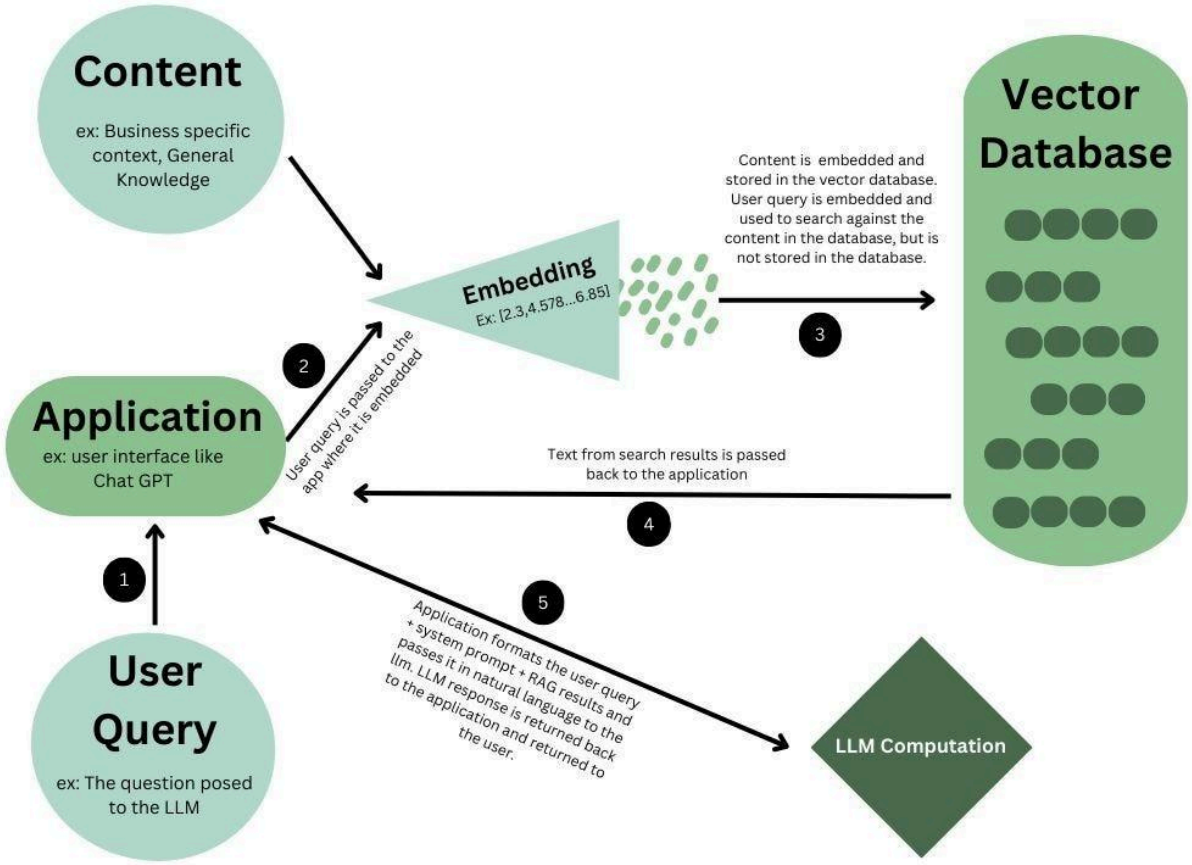
Day 25

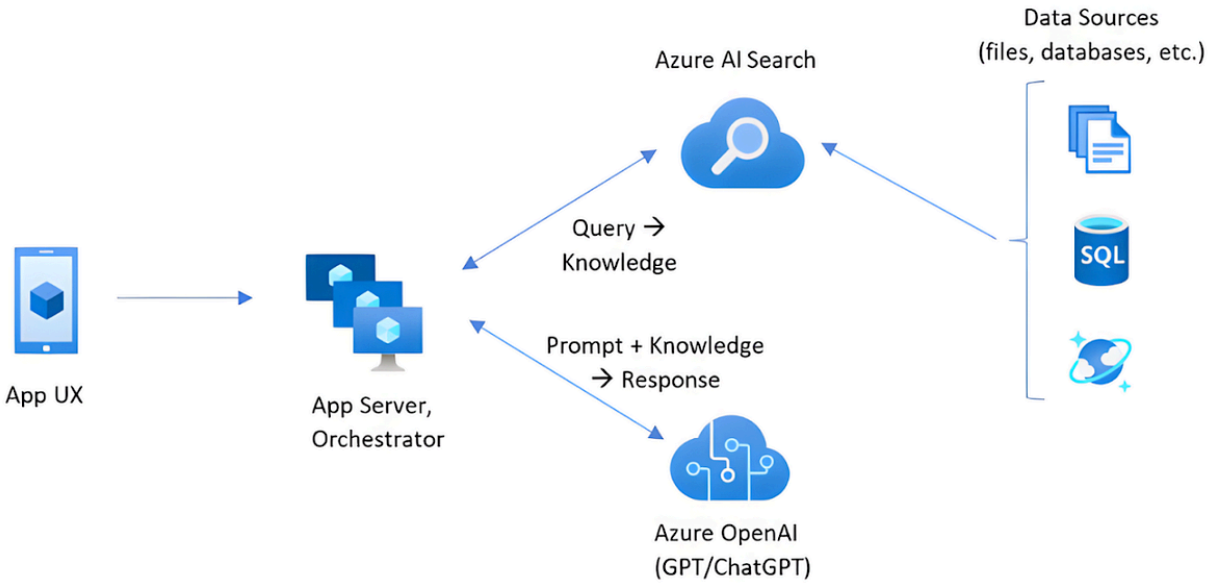
- Prompt evaluation & improvements

🔹 RAG (Retrieval Augmented Generation) (Days 26–35)



created by Sandi Besen





Day 26

- What is embeddings
- Why RAG is needed

Day 27

- Text chunking strategies

Day 28

- Vector databases concept

Day 29

- Store embeddings

Day 30

- Similarity search

Day 31

- Inject retrieved context into prompt

Day 32

- RAG response tuning

Day 33

- Handle large PDFs

Day 34

- RAG caching strategies

Day 35

- Mini project:
  👉 Upload PDF → ask questions → AI answers

🟢 DAYS 36–50: AI + Backend + Frontend Integration

Backend (Laravel / Node)

Day 36

- AI service layer design

Day 37

- Async jobs for AI calls

Day 38

- Rate limiting & cost control

Day 39

- AI request logging

Day 40

- Error handling & fallbacks

Frontend (React / Next.js)

Day 41

- Chat UI design

Day 42

- Streaming responses

Day 43

- AI loading states

Day 44

- Prompt input UX

Day 45

- Feedback buttons (👍 👎)

Full Integration

Day 46

- End-to-end AI flow

Day 47

- Secure API keys

Day 48

- Environment configs

Day 49

- Performance optimization

Day 50

- AI audit & logs dashboard

🟢 DAYS 51–60: Real AI Projects (Resume Ready)

Project 1 (Days 51–54)

AI Medical Notes Assistant

- Upload notes
- AI summary
- Suggested codes

Project 2 (Days 55–57)

AI Document Q&A

- PDF upload
- Ask questions
- Structured answers

Project 3 (Days 58–60)

AI Report Generator

- Raw data → executive summary

2️⃣ Exact Tools & Libraries (NO CONFUSION)

🔹 AI / LLM

- OpenAI / Azure OpenAI
- Prompt templates

🔹 Backend

- Laravel (AI microservice)
- Node.js (optional)
- Queue jobs (Redis / DB)

🔹 Python

- requests
- pydantic (response validation)

🔹 Vector Database

- FAISS (local)
- Pinecone / Weaviate (prod)

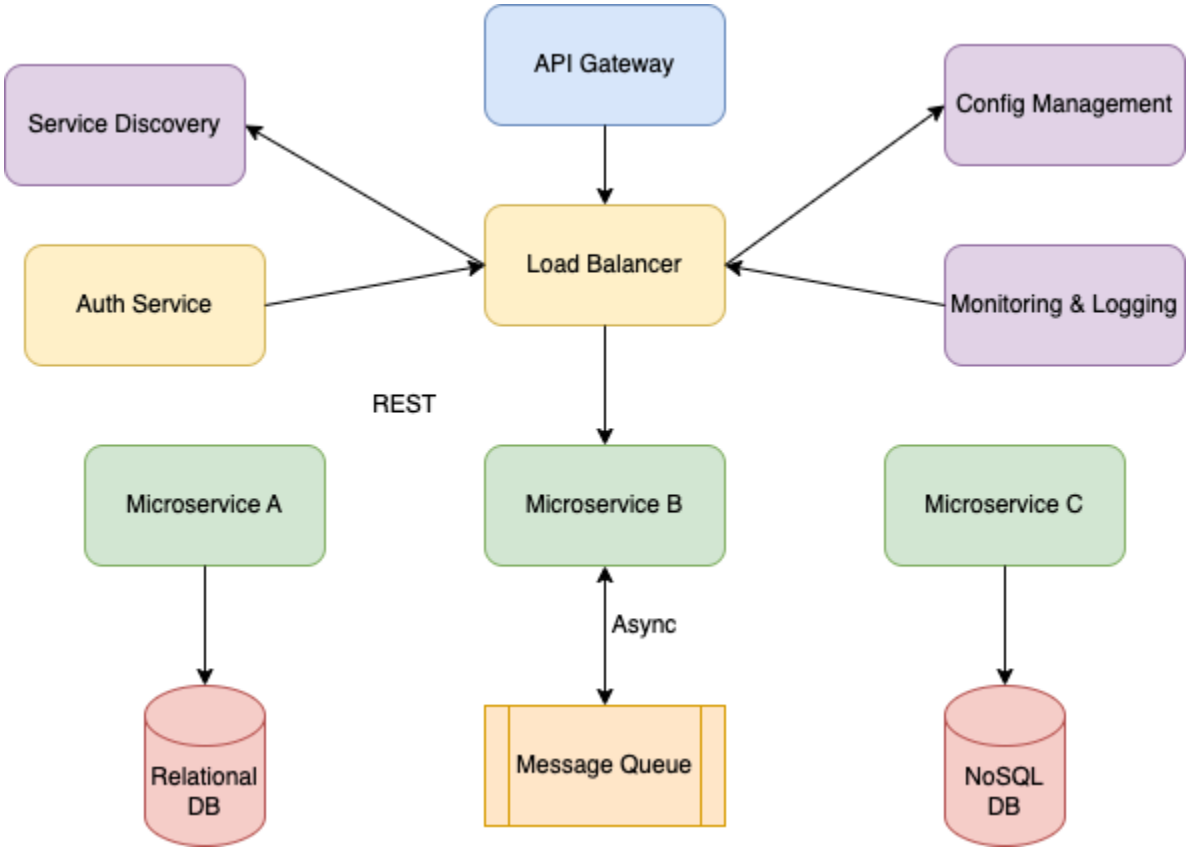
🔹 Frontend

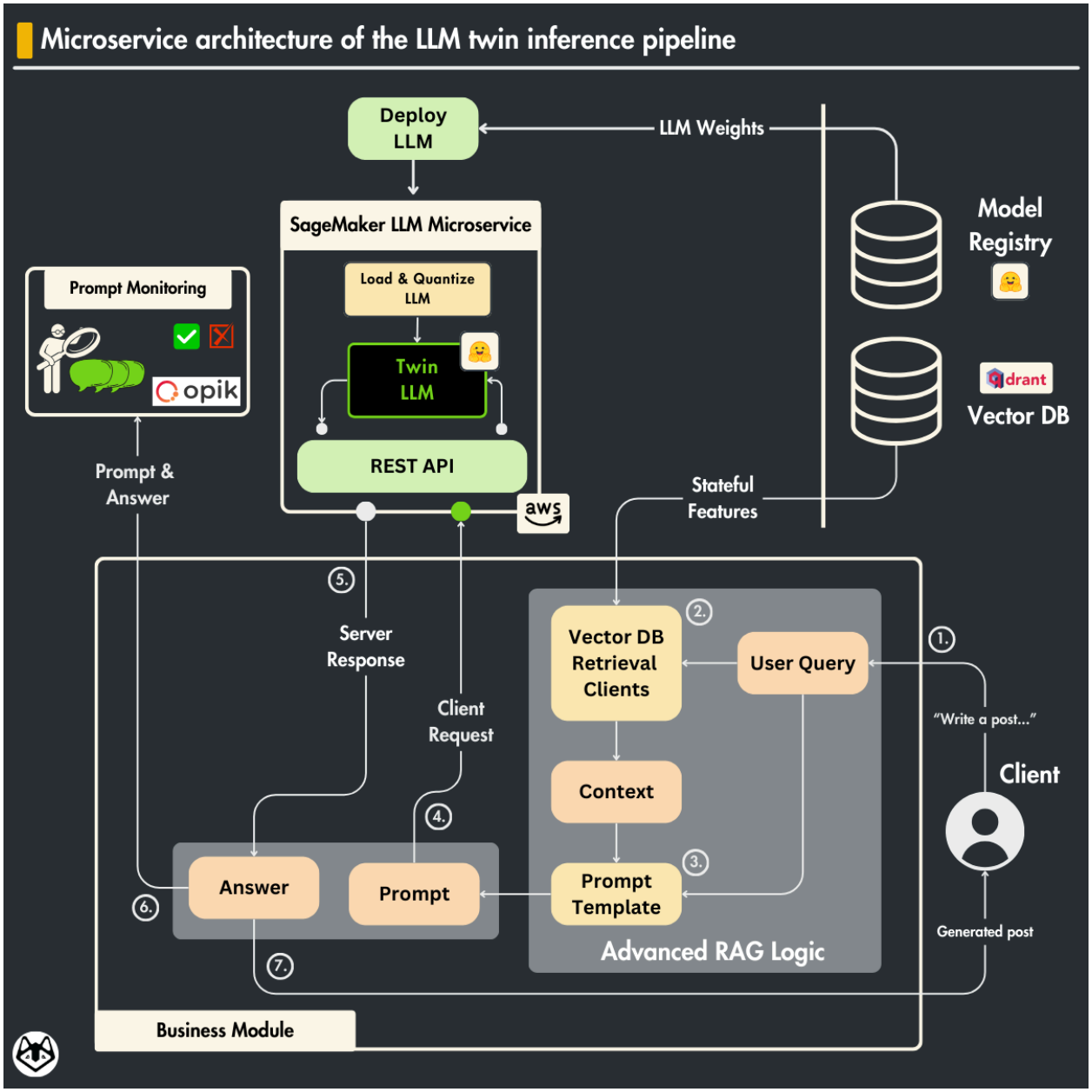
- Next.js
- React
- Tailwind (UI)

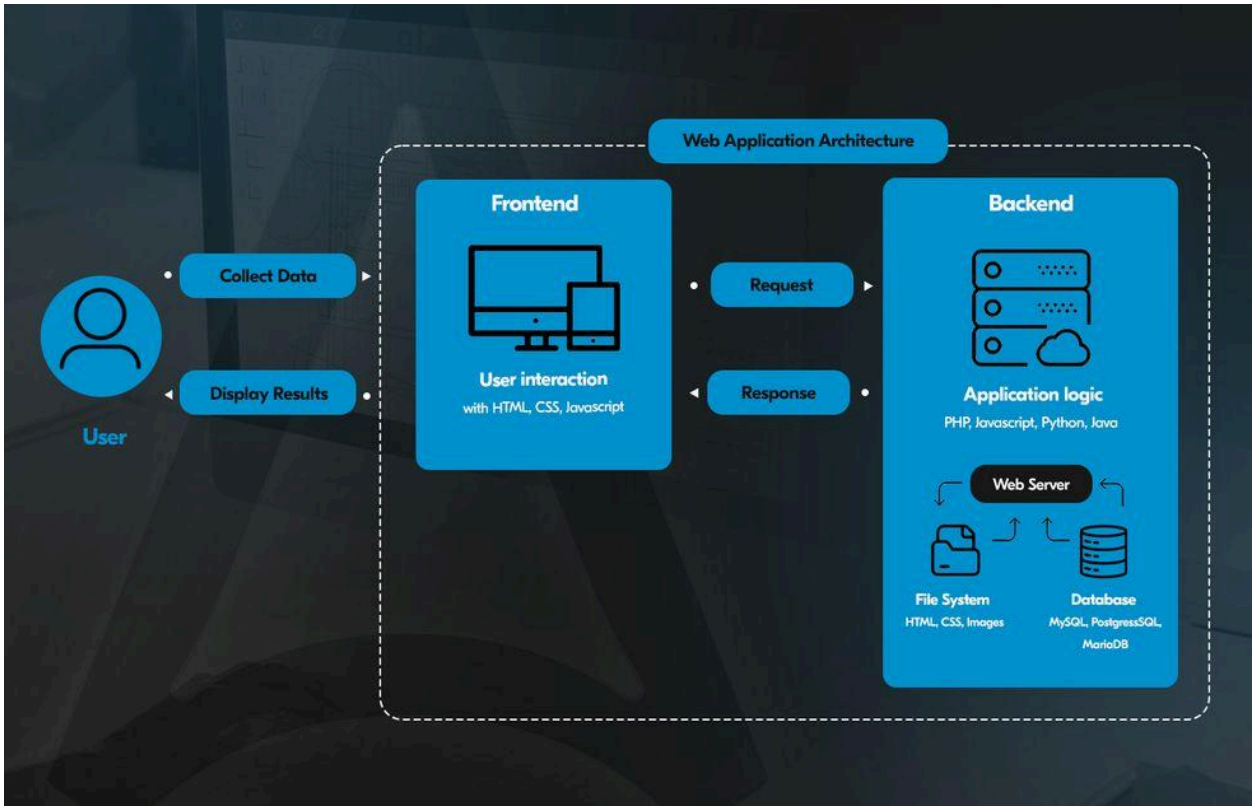
🔹 DevOps

- Docker
- Environment secrets
- Logging tools

3️⃣ AI Project Architecture (Production Grade)




Microservice architecture of the LLM twin inference pipeline
Deploy LLM
LLM Weights
Model Registry
SageMaker LLM Microservice
Load & Quantize LLM
Twin LLM
REST API
Prompt Monitoring
opik
Vector DB
Prompt & Answer
Stateful Features
5.
Server Response
Client Request
2.
Vector DB Retrieval Clients
User Query
1.
"Write a post..."
Client
Context
4.
3.
Answer
Prompt
Prompt Template
6.
Advanced RAG Logic
Generated post
7.
Business Module



🔹 High-Level Architecture

```
Frontend (Next.js)
  |
  v
Backend API (Laravel)
  |
  |--> AI Service Layer
  |       |
  |       |--> Prompt Engine
  |       |--> RAG Engine
  |       |--> LLM API
  |
  |--> Vector DB
  |
  |--> Logs & Monitoring
```

🔹 Request Flow

1. User enters query

2. Backend validates input
3. RAG fetches relevant context
4. Prompt builder injects context
5. LLM generates response
6. Response validated
7. Result sent to UI
8. Logs stored

🔹 Folder Structure (Laravel Example)

```
app/
├── AI/
│   ├── PromptBuilder.php
│   ├── LLMClient.php
│   ├── RAGService.php
│   └── ResponseValidator.php
├── Jobs/
├── Controllers/
```

🎯 Final Reality Check

After this plan, you can confidently say:

✅ “I am an AI Engineer”
✅ “I build AI-powered products”
✅ “I integrate LLMs into real systems”

Absolutely — here's a **complete, practical roadmap**:
📍 **Learn Python + FastAPI + AI engineering**
📍 **Build portfolio projects**
📍 **Find remote jobs globally**
📍 **Crack interviews for high-paying remote & FAANG roles (₹70 LPA+ / $60k+)**

I'll break this into **5 sections** with clear steps you can start following today.

🎯 1. Your Goal (Remote + FAANG + ₹70 LPA+)

Remote and global opportunities pay **significantly higher than local Indian offers** (often $60–100k+ / equivalent ₹50–90 LPA+) especially in AI-focused roles. You're aiming for either:

- Fully **remote jobs**
- **Remote-friendly global companies**
- FAANG (though many big FAANG teams require hybrid onsite at some stage) (Reddit)

This means you must prepare not only technically but also for a global interview process.

🧱 2. Core Skill Path (Python + FastAPI + AI)

📌 Immediate Priorities

A. Python Fundamentals

- Language basics (loops, functions, OOP)
- Intermediate: async, data modelling
- Modules: `requests`, `logging`, `typing`

B. FastAPI

- Routing & APIs
- Request-response
- Dependency injection
- Authentication
- Async behaviours

C. AI Engineering

- Calling LLM APIs (OpenAI, Azure)

- Embeddings + RAG systems
- Prompt engineering
- Building microservices for AI

Focus: Build real code, not just read.

🛠️ 3. Project Roadmap (Portfolio Focus)

Your goal is **projects that prove capability** to hiring managers:

🔹 Project 1 — AI Document Q&A

- Upload PDF / text corpora
- Build embeddings
- RAG + FastAPI endpoint
- Frontend React/Next for chat UI

Skills Proven:
Python, FastAPI, AI, RAG, database

🔹 Project 2 — AI Notes Summarizer (Healthcare focus optional)

- Upload medical notes
- Summaries & structured output (JSON)
- Suggest codes or insights

Skills Proven:
LLM handling, output engineering, error control

🔹 Project 3 — End-to-End Product Backend

- Multi-service architecture
- Auth (JWT)
- Logging, retries, cost control

Skills Proven:
Backend architecture + SaaS readiness

💼 4. Where to Find Remote Jobs

You need **multiple channels**:

🔹 1. Remote Job Boards

These list **remote Python & AI roles** globally.

- **RemoteOK** — Remote Python/AI listings (US & worldwide) (Remote OK)
- **Arc.dev** — Senior remote FastAPI & Python jobs (global) (Arc)
- **CutShort** — Remote Python jobs with filters (Cutshort)
- **Instahyre** — Remote FastAPI / backend listings (Instahyre)

  Set alerts on these platforms for keywords like:
  Python, *FastAPI*, *AI Engineer*, *Machine Learning backend*, *Generative AI engineer*.

🔹 2. Freelance & Marketplace (for early wins)

- **Toptal** — Top-tier remote contractor market (strict vetting but high pay) (Wikipedia)
- **Andela Marketplace** — Matches talent with global remote engagements (Wikipedia)

These can give you high-paying contracts while you build towards full-time offers.

🔹 3. LinkedIn + GitHub

- Collect leads, connect with engineers and recruiters
- Publish projects with detailed READMEs
- Use keywords like *FastAPI*, *Generative AI Backend*, *RAG*

Recruiters often search LinkedIn first.

🧠 5. Interview Prep Workflow

📌 Step 1 — Technical Fundamentals

Focus on core competencies:

- **Data Structures & Algorithms**
- **System design fundamentals**

- **Python and service design**

FAANG-style interviews test these heavily — coding + design rounds. (Interview Kickstart)

📌 Step 2 — Remote Interview Practice

Remote interviews are almost the same as onsite interviews: they include:

- Recruiter screens
- Coding challenges
- System design / architecture rounds
- Behavioral rounds (We Work Remotely)

📌 Step 3 — Mock Interviews

Platforms for practice:

- LeetCode (Coding)
- HackerRank (Coding + assignments)
- Interview Kickstart style study scripts (system design & interview patterns) (Interview Kickstart)

📌 Step 4 — Portfolio + Resume Sync

Your resume must NOT read like a standard Laravel resume. Instead:

✅ Highlight backend + FastAPI experience
✅ Show real AI project links (GitHub live links)
✅ Mention specific AI problems solved (RAG, embeddings, prompt engineering)

Recruiters want to **see demonstrable skills before interviews**. (Medium)

📆 Typical Remote Job Application Funnel

1. **Discover jobs** → reminders & alerts
2. **Tailor resume + LinkedIn** → role-specific keywords
3. **Apply early + wide** → global range

4. **Recruiter screen** → communication + fit
5. **Technical coding screen**
6. **System design + AI domain discussion**
7. **Offer negotiation**

Remote hiring is competitive; apply *frequently* and don't wait for perfect preparation.

🧠 Final Strategies for ₹70 LPA+ Targets

🚀 Target Roles

- AI Backend Engineer
- Generative AI Engineer
- Python/Fullstack with AI focus
- Senior FastAPI + AI Platform Engineer

📌 Why This Works

These roles are in higher demand — companies pay premium for:
✔ Real AI product experience
✔ Strong backend + API design
✔ Good coding + architecture skills
✔ Remote readiness

Remote jobs often pay globally x1.5–x2 India CTC if you qualify.

📌 Quick Job Alerts You Should Set Today

- "Remote Python FastAPI AI"
- "Generative AI backend engineer remote"
- "Remote backend engineer Python microservices"

Set alerts on RemoteOK, Arc.dev, Cutshort, Instahyre.

🚀 When To Start Applying?

Apply **once your first 2 projects are functional** with a polished GitHub + live demo link — this is usually within **45–60 days** once you follow the roadmap.
